Modifying deployment descriptor (/WEB-INF/web.xml)

The configuration shown below must be added in the deployment descriptor. Add *applicationContext.xml*, *hdiv-config.xml* and *hdiv-validation.xml* to the web.xml file, as shown below:

```
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>
        /WEB-INF/applicationContext.xml,/WEB-INF/hdivconfig.xml,
/WEB-INF/hdiv-validations.xml
    </param-value>
</context-param>
```

Add HDIV initialisation listeners as follows:

```
<listener>
      <listener-class>
          org.hdiv.listener.InitListener
      </listener-class>
</listener>
```

Defining validation filters

Validation filters are to be defined before any *struts2* filter and *org.apache.struts2.dispatcher.FilterDispatcher* class, in order to ensure their execution before any Struts operation:

```
<filter>
        <filter-name>ValidatorFilter</filter-name>
        <filter-class>org.hdiv.filter.ValidatorFilter</filter-
class>
</filter>

<filter-mapping>
         <filter-name>ValidatorFilter</filter-name>
         <url-pattern>*.action</url-pattern>
</filter-mapping>

<filter-mapping>
         <filter-name>ValidatorFilter</filter-name>
         <url-pattern>*.jsp</url-pattern>
</filter-mapping>

<filter-mapping>
         <filter-name>ValidatorFilter</filter-name>
         <url-pattern>*.do</url-pattern>
</filter-mapping>
```

The above configuration will provide validation filters for the extensions of all possible actions, and for the JSP pages.

Now, in the Struts2 filter, add the following configuration lines:

```
<filter>
        <filter-name>struts2</filter-name>
        <filter-class>
            org.apache.struts2.dispatcher.FilterDispatcher
      </filter-class>
        <init-param>
                <param-name>config</param-name>
                <param-value>hdiv-default.xml,struts-plugin.
xml,struts.xml</param-value>
        </init-param>
</filter>
```

Note: The given installation steps may differ if you are using the Spring framework also, which will require some additional changes in configuration files. For more on this, please refer to the links at the end of this article.

Let's now configure the operational strategy, discussed earlier. The *hdiv-config.xml* comes with the memory strategy as default in the strategy bean:

```
<bean id="strategy" class="java.lang.String">
            <constructor-arg>
                    <value>memory</value>
            </constructor-arg>
</bean>
```

It can be changed, however, to *cipher* or *hash*.

To activate confidentiality, configure as shown below:

```
<bean id="confidentiality" class="java.lang.String">
            <constructor-arg>
                    <value>true</value>
            </constructor-arg>
</bean>
```

To deactivate it, just set it to false.

The HDIV logger

HDIV also provides logging functionality, which can help administrators detect attacks and other invalid actions performed on the Web application. By default, HDIV is designed to use *log4j*, and its properties are defined under the *log4j.properties* file in HDIV. The attack message logs will be written in *c:/hdiv.log;* however you can also change this path in *log4j.properties.*

The log format in which events are logged is:

```
[type of attack];[action];[parameter];[value];[userLocalIP];[IP]
;[userId]
```

These fields have the following data:

- *[type of attack]:* This denotes whether any action, parameter, parameter value, pageID, cookie or _HDIV_ STATE value is invalid.